

AI Foundations & Safety Study Plan

A Discipline-First Roadmap to AI Safety Research Competence

Third Iteration — 42 Weeks (~10.5 Months)

This plan is designed for 1 hour of focused AI study per day, alongside a separate 1-hour daily block for CPTS certification work. The plan is structured around 42 weeks (~10.5 months), broken into eight phases that build mathematical fluency, implementation skill, research intuition, and deep AI safety expertise.

Plan Architecture

Phase	Weeks	Focus	Outcome
1	1–6	Math Foundations & NumPy Fluency	Comfortable with linear algebra, calculus, probability in code
2	7–12	ML Algorithms from Scratch	Personal library of implemented ML algorithms
3	13–19	Deep Learning, Tokenization, Transformers & Systems	Can write a transformer from a blank file; understands training infrastructure
4	20–23	Adversarial ML & Model Security	Bridges cybersecurity background with AI
5	24–29	ARENA, Experimental Methods & Causal Inference	Independent interpretability research with rigorous methodology
6	30–34	Information Theory, Representations & Training Dynamics	Formal toolkit for analyzing model internals and learning phenomena
7	35–38	RL, RLHF, LoRA & the Alignment Pipeline	Understands how modern LLMs are trained, fine-tuned, and aligned
8	39–42	AI Safety Theory, Red-Teaming & Evals	Research-ready AI safety practitioner

Core Principles

- **Build, don't memorize.** Every concept gets implemented in code. If you can't rebuild it from scratch in NumPy or PyTorch, you don't understand it yet.
- **Progressive overload for your brain.** You start at 1 hour/day. By Phase 3, the plan introduces optional extension blocks to let you naturally expand to 1.5–2 hours as your stamina grows. Never force it—consistency beats intensity.
- **Interleave theory and practice.** Each session is split: ~25 minutes of conceptual input (video/reading), ~30 minutes of implementation, ~5 minutes of reflection/notes.
- **Weekly checkpoints.** Every Friday is a “Review & Build” day where you consolidate the week by building something small from scratch without any references.

- **Safety thinking from the start.** Even in Phases 1–3, you’ll develop the habit of asking “what could go wrong?” and “what assumptions is this model making?” This adversarial mindset is your cybersecurity background applied to AI.

Session Template (1 Hour)

Block	Duration	Activity
Conceptual Input	25 min	Watch lecture segment, read textbook section, or work through derivation on paper
Implementation	30 min	Code what you just learned from scratch. No copy-paste. Close the tutorial.
Reflection	5 min	Write 2–3 sentences in study journal: what clicked, what’s still fuzzy, what to revisit.

Note on the study journal: Keep a simple markdown file or notebook. This is not busywork—the act of articulating confusion is one of the most effective learning techniques available. Over time, this journal becomes a personalized reference that’s vastly more useful than any textbook.

Phase 1: Mathematical Intuition & NumPy Fluency

Weeks 1–6 | Building the Foundation

Goal: Develop geometric intuition for linear algebra and calculus, and become fluent in NumPy as your “math scratchpad.” By the end of Phase 1, you should be able to implement matrix operations, derivatives, and gradient descent from scratch without referencing documentation.

Resources

- **3Blue1Brown — Essence of Linear Algebra** (16 videos, ~3.5 hours total)
- **3Blue1Brown — Essence of Calculus** (12 videos, ~3 hours total)
- **Mathematics for Machine Learning (MML)** by Deisenroth, Faisal & Ong — Chapters 2–5 (free PDF). This is a better fit than PRML at this stage because it’s explicitly designed as a bridge between undergrad math and ML. PRML assumes you already have this bridge.
- **NumPy documentation** — specifically the broadcasting rules and array manipulation pages

Week 1: Vectors, Spaces & NumPy Foundations

Theme: What are vectors, really? And how do we compute with them?

Day	Conceptual	Implementation	Reflection
Mon	3B1B Essence of Linear Algebra: Ep 1 “Vectors, what even are they?” + Ep 2 “Linear combinations, span, basis”	Create vectors.py. Implement a Vector class from scratch with <code>__add__</code> , <code>__sub__</code> , <code>scalar_multiply</code> , <code>dot_product</code> , <code>magnitude</code> . No NumPy yet.	Journal: How does 3B1B’s geometric view differ from how you thought about vectors?
Tue	3B1B: Ep 3 “Linear transformations and matrices”	Extend your Vector class: implement a Matrix class with <code>multiply</code> (matrix-vector), <code>transpose</code> , and <code>identity</code> . Test with 2D rotation matrices.	What does a matrix “do” geometrically?
Wed	MML Chapter 2, Sections 2.1–2.3 (Vector Spaces, Linear Independence)	Rewrite Monday’s Vector class using NumPy arrays. Compare: measure execution time of your pure Python vs NumPy for 10,000 dot products.	Where does NumPy’s speed come from?
Thu	3B1B: Ep 4 “Matrix multiplication as composition” + Ep 5 “Three-dimensional transformations”	Build a 2D transformation visualizer: use matplotlib to show how a grid of points transforms under different 2x2 matrices (rotation, shear, scaling, projection).	Which transformations preserve area? Which don’t?
Fri	— Review Day —	FROM SCRATCH CHALLENGE: Without any references, implement matrix multiplication for arbitrary-sized matrices in	What was hardest to recall?

		pure Python. Then verify against <code>np.matmul</code> . If stuck, that's your signal for what to revisit next week.	
Sat (opt)	MML Chapter 2, Sections 2.4–2.6 (Basis, Rank, Linear Mappings)	Implement a function that determines if a set of vectors is linearly independent using row reduction.	
Sun	Rest or light review of journal notes		

Week 2: Determinants, Inverses & Solving Systems

Theme: When can we reverse a transformation? What does a determinant tell us?

Day	Conceptual	Implementation	Reflection
Mon	3B1B: Ep 6 “Determinant” + Ep 7 “Inverse matrices, column space, null space”	Implement determinant computation for 2x2 and 3x3 matrices from scratch. Then implement 2x2 matrix inverse. Verify with <code>np.linalg.det</code> and <code>np.linalg.inv</code> .	When does a matrix have no inverse? What happens geometrically?
Tue	3B1B: Ep 8 “Nonsquare matrices” + Ep 9 “Dot products and duality”	Implement Gaussian elimination from scratch to solve $Ax = b$. Test on 3 systems: one with unique solution, one with no solution, one with infinite solutions.	How does elimination relate to row reduction you see in textbooks?
Wed	MML Chapter 2, Sections 2.7–2.9 (solving linear systems)	Extend your Gaussian elimination to compute the inverse of an arbitrary NxN matrix by augmenting with identity. Compare speed vs <code>np.linalg.inv</code> .	What's the computational cost of inversion?
Thu	3B1B: Ep 10 “Cross products” + Ep 11 “Cramer's rule”	Build a linear system solver that detects the type of solution (unique, none, infinite) and returns the solution or characterizes the solution space.	When would you use Cramer's rule vs elimination in practice?
Fri	— Review Day —	FROM SCRATCH CHALLENGE: Given a random 4x4 matrix, compute its determinant, inverse (if exists), and solve $Ax = b$ for a random b . No references.	Grade yourself: which operations are automatic vs still require thinking?
Sat (opt)	MML Chapter 3, Sections 3.1–3.3 (Norms, Inner Products)	Implement L1, L2, and L-infinity norms. Implement cosine similarity. Build a function that finds the nearest neighbor to a query vector in a dataset.	
Sun	Rest		

Week 3: Eigenvalues, Eigenvectors & PCA

Theme: The most important decomposition in all of ML.

Day	Conceptual	Implementation	Reflection
Mon	3B1B: Ep 14 “Eigenvectors and eigenvalues”	Implement the power iteration method to find the dominant eigenvector of a matrix. Compare with <code>np.linalg.eig</code> .	What does an eigenvector “mean” geometrically? Why does ML care?
Tue	MML Chapter 4, Sections 4.1–4.3 (Determinants, Eigenvalues, Eigendecomposition)	Implement eigendecomposition: given eigenvalues and eigenvectors, reconstruct the original matrix. Verify $A = PDP^{-1}$.	What types of matrices are diagonalizable?
Wed	MML Chapter 4, Sections 4.4–4.5 (SVD)	Implement SVD from scratch for a 2x3 matrix using the eigendecomposition of $A^T A$. Compare with <code>np.linalg.svd</code> .	How does SVD relate to eigendecomposition? When do you use which?
Thu	MML Chapter 10, Sections 10.1–10.3 (PCA derivation)	Implement PCA from scratch: center data, compute covariance matrix, find eigenvectors, project. Test on a synthetic 3D dataset projected to 2D. Visualize.	Why does PCA use the covariance matrix?
Fri	— Review Day —	FROM SCRATCH CHALLENGE: Load the Iris dataset (<code>sklearn.datasets</code>). Implement PCA from scratch to reduce from 4D to 2D. Plot clusters. Compare with <code>sklearn.decomposition.PCA</code> .	How much variance did your first 2 components capture?
Sat (opt)	3B1B: Ep 15 “Abstract vector spaces” + review Chapter 4 remainder	Implement a function that compresses an image using SVD truncation. Show the image at rank 1, 5, 10, 50.	
Sun	Rest		

Week 4: Calculus — Derivatives & the Chain Rule

Theme: Calculus is about local linear approximation. That’s the secret to all of deep learning.

Day	Conceptual	Implementation	Reflection
Mon	3B1B Essence of Calculus: Ep 1 “The essence of calculus” + Ep 2 “The paradox of the derivative”	Implement numerical differentiation: write a function <code>numerical_derivative(f, x, h=1e-7)</code> that approximates df/dx . Test on x^2 , $\sin(x)$, e^x . Compare with analytical derivatives.	Why does $h=1e-7$ work better than $h=1e-15$?
Tue	3B1B: Ep 3 “Derivative formulas through geometry” + Ep 4	Implement an Expression class that supports automatic symbolic differentiation for $+$, $*$, power, sin,	How does the chain rule work as “multiplication of local stretching factors”?

	“Visualizing the chain rule”	cos, exp. Build a tiny computer algebra system.	
Wed	3B1B: Ep 5–7 (Euler’s number, implicit differentiation, limits)	Implement numerical gradient for multivariate functions: <code>compute_gradient(f, point, h=1e-7)</code> that returns the gradient vector. Test on $f(x,y) = x^2 + 3xy + y^2$.	What is a gradient geometrically?
Thu	MML Chapter 5, Sections 5.1–5.3 (Gradients, Jacobians)	Implement the Jacobian matrix computation for a vector-valued function $f: \mathbb{R}^2 \rightarrow \mathbb{R}^3$. Verify numerically.	When do you need the Jacobian vs just the gradient?
Fri	— Review Day —	FROM SCRATCH CHALLENGE: Implement gradient descent to find the minimum of $f(x,y) = (x-3)^2 + (y+1)^2 + 2*\sin(x*y)$. Visualize the optimization path on a contour plot.	Did gradient descent find the global minimum? Why or why not?
Sat (opt)	3B1B: Ep 8–11 (Integration series — lighter material, good for intuition)	Implement numerical integration using the trapezoidal rule and Simpson’s rule. Compare accuracy.	
Sun	Rest		

Week 5: Multivariable Calculus & Optimization

Theme: How gradient descent actually works, and why it’s the engine of all neural network training.

Day	Conceptual	Implementation	Reflection
Mon	MML Chapter 5, Sections 5.4–5.6 (Backpropagation intuition, higher-order derivatives)	Implement gradient descent with configurable learning rate and momentum. Test on Rosenbrock’s function. Visualize convergence.	How does learning rate affect convergence?
Tue	MML Chapter 7, Sections 7.1–7.2 (Optimization: gradient descent, convergence)	Implement three variants: vanilla GD, GD with momentum, and Nesterov accelerated gradient. Compare convergence curves on a quadratic bowl.	Why does momentum help?
Wed	MML Chapter 7, Section 7.3 (Stochastic gradient descent)	Implement SGD on a synthetic dataset: generate 1000 points from $y = 3x + 2 + \text{noise}$. Compare full-batch GD vs mini-batch SGD (batch size 32) vs single-sample SGD.	Why is SGD noisier but often faster?
Thu	Khan Academy or StatQuest: “Partial	Implement Adam optimizer from scratch (maintain running averages of first and second	What problem does Adam solve that vanilla SGD doesn’t?

	derivatives” and “Gradient” videos	moments). Compare with your SGD on the same dataset.	
Fri	— Review Day —	FROM SCRATCH CHALLENGE: Implement linear regression using gradient descent from scratch. No sklearn, no np.linalg.lstsq. Generate synthetic data, fit it, plot the learned line.	Does your GD solution match the closed-form solution?
Sat (opt)	3B1B: “Neural Networks” series Ep 1–2 (gentle intro)	Extend your linear regression to polynomial regression by feature engineering. Fit degree-2, 5, 10 polynomials. Observe overfitting.	
Sun	Rest		

Week 6: Probability & Statistics for ML

Theme: Uncertainty is not a bug—it’s the fundamental language of machine learning.

Day	Conceptual	Implementation	Reflection
Mon	StatQuest: “Probability vs Likelihood” + “Maximum Likelihood Estimation”	Implement MLE for a Gaussian: given N samples, compute the ML estimates of mean and variance from scratch. Compare with np.mean and np.var.	What’s the difference between probability and likelihood?
Tue	MML Chapter 6, Sections 6.1–6.3 (Probability distributions, Bayes’ theorem)	Implement Bayes’ theorem for a medical test scenario: given sensitivity, specificity, and base rate, compute posterior probability of disease. Explore how base rate affects results.	Why is Bayes’ theorem so counterintuitive?
Wed	StatQuest: “Gaussian (Normal) Distribution” + “PCA main ideas”	Implement a Gaussian class: pdf, log_pdf, sample, and fit (MLE). Implement a mixture of 2 Gaussians and visualize.	Why are Gaussians so prevalent in ML?
Thu	MML Chapter 6, Sections 6.4–6.7 (Exponential family, sufficient statistics, conjugate priors)	Implement the EM algorithm for a 2-component Gaussian Mixture Model from scratch. Test on synthetic data with two clusters.	Why can’t we just use MLE directly for mixtures?
Fri	— Review Day —	PHASE 1 CAPSTONE: Implement logistic regression from scratch using gradient descent with log-likelihood loss. Train on a synthetic 2-class dataset. Plot the decision boundary.	Write a one-page summary of what you learned in Phase 1.
Sat (opt)	Review: skim MML Chapters 2–7 summaries.	Revisit any implementation from the past 6 weeks that you struggled with. Rebuild it.	

	Identify any remaining gaps.		
Sun	Rest		

Phase 2: Machine Learning from Scratch

Weeks 7–12 | From Theory to Working Models

Goal: Implement the core ML algorithms from scratch in NumPy, understanding every line of math. By the end, you'll have a personal library of ML implementations that you truly own intellectually.

Resources

- **Andrew Ng — Machine Learning Specialization** (Coursera, Course 1 only: “Supervised Machine Learning”). Use this as your conceptual backbone, but always re-implement from scratch rather than using the provided starter code.
- **StatQuest with Josh Starmer** (YouTube) — Watch the corresponding StatQuest video for every algorithm you implement. His visual explanations are the best quick-reference for intuition.
- **ISLR Chapter 2–4** (free PDF) — Read alongside for statistical framing.

Discipline Note: Expanding Your Sessions

By now you've been studying consistently for 6 weeks. If you're finding 1 hour feels short and you want more, **begin adding a 15-minute optional extension** on days when you're in a flow state. Don't force it. The goal is 1 hour minimum, 1 hour 15 minutes when it feels natural. By Week 10, you can aim for 1.5-hour sessions on weekdays.

Week 7: Linear Regression — The Full Picture

Day	Conceptual	Implementation	Reflection
Mon	Ng: Supervised ML Week 1 (intro, cost function)	Build a LinearRegression class from scratch with fit (using normal equation AND gradient descent), predict, and score (R^2).	Why do the normal equation and GD give the same answer? When would you prefer one over the other?
Tue	StatQuest: “Linear Regression” + “R-squared”	Add regularization to your class: L2 (Ridge) and L1 (Lasso). Implement the closed-form Ridge solution and gradient-based Lasso.	What does regularization do geometrically?
Wed	ISLR Chapter 3 (Linear Regression) — focus on Sections 3.1–3.2	Implement polynomial feature expansion and cross-validation (k-fold) from scratch. Show overfitting/underfitting curves.	How does degree relate to model complexity?
Thu	Ng: Supervised ML Week 2 (multiple regression, feature scaling)	Implement feature scaling (z-score, min-max) from scratch. Show how scaling affects GD convergence speed on a dataset with features of very different magnitudes.	Why does feature scaling matter for GD but not for the normal equation?
Fri	— Review Day —	FROM SCRATCH: Load the California Housing dataset.	Compare your results with sklearn's Ridge.

		Implement the full pipeline: load data, scale features, split train/test, fit with Ridge regression, evaluate. No sklearn.	
--	--	--	--

Week 8: Classification — Logistic Regression & Evaluation

Day	Conceptual	Implementation	Reflection
Mon	Ng: Supervised ML Week 3 (logistic regression, decision boundary)	Build a LogisticRegression class: sigmoid, compute_loss (cross-entropy), fit (GD), predict, predict_proba.	Why cross-entropy and not MSE for classification?
Tue	StatQuest: “Logistic Regression” + “ROC and AUC”	Implement evaluation metrics from scratch: accuracy, precision, recall, F1, confusion matrix. Plot ROC curve using your predict_proba.	Why is accuracy alone misleading on imbalanced data?
Wed	ISLR Chapter 4, Sections 4.1–4.3 (Classification)	Implement multi-class logistic regression using one-vs-rest and softmax approaches. Test on Iris dataset.	What’s the difference between one-vs-rest and softmax?
Thu	StatQuest: “Regularization” + “Cross Validation”	Add L2 regularization to your logistic regression. Implement a hyperparameter search over regularization strength using cross-validation.	How do you select the regularization strength in practice?
Fri	— Review Day —	FROM SCRATCH: Binary classification pipeline on a real dataset (e.g., Breast Cancer Wisconsin). Feature scaling, train/val/test split, logistic regression, full evaluation report.	

Week 9: Decision Trees & Ensembles

Day	Conceptual	Implementation	Reflection
Mon	StatQuest: “Decision Trees” + “Gini Impurity vs Entropy”	Implement a DecisionTreeClassifier from scratch: Node class, Gini impurity, recursive splitting, prediction.	When does a tree stop splitting?
Tue	StatQuest: “Random Forests”	Implement Random Forest from scratch: bootstrap sampling, feature subsampling, majority vote aggregation.	Why does averaging multiple trees reduce overfitting?
Wed	StatQuest: “Gradient Boost” Parts 1–2	Implement a simple gradient boosted regressor: sequential fitting of shallow trees to residuals.	How is boosting fundamentally different from bagging?

Thu	ISLR Chapter 8 (Tree-Based Methods) — skim Sections 8.1–8.2	Add a <code>max_depth</code> parameter and <code>min_samples_split</code> to your decision tree. Visualize how tree depth affects the decision boundary on a 2D dataset.	How does tree depth relate to bias/variance?
Fri	— Review Day —	FROM SCRATCH: Compare your decision tree, random forest, and gradient boosting on a classification dataset. Plot learning curves.	Which model generalizes best and why?

Weeks 10–11: Neural Networks from Scratch

This is the critical bridge to deep learning. Take two full weeks.

Day	Conceptual	Implementation	Reflection
W10 Mon	3B1B Neural Networks: Ep 1–2 (structure, learning)	Build a Neuron class: weighted sum, bias, activation (sigmoid). Build a Layer class: array of neurons.	What computation does a single neuron perform?
W10 Tue	3B1B Neural Networks: Ep 3 (backpropagation)	Implement forward pass for a 2-layer network (input → hidden → output). Hardcode weights, verify outputs match hand computation.	Trace the dimensions through each layer on paper.
W10 Wed	3B1B Neural Networks: Ep 4 (calculus of backpropagation)	Derive and implement backpropagation for your 2-layer network on paper first, then in code. Compute gradients for each weight.	What does the chain rule compute at each layer?
W10 Thu	Ng: Advanced Learning Algorithms Week 1 (neural network intro)	Refactor into a clean NeuralNetwork class with: <code>add_layer</code> , <code>forward</code> , <code>backward</code> , <code>update_weights</code> . Train on XOR.	Why can't a single-layer network solve XOR?
W10 Fri	— Review Day —	FROM SCRATCH: Train your neural network on MNIST digits (flatten to 784-dim). Achieve >90% accuracy.	
W11 Mon	Karpathy: Zero to Hero, Video 1: “micrograd” (first 45 min—split across 2 days)	Begin implementing Karpathy's Value class with gradient tracking. Focus on forward pass and basic operations (+, *).	How does automatic differentiation differ from numerical?
W11 Tue	Karpathy: Video 1 (remaining ~45 min)	Complete micrograd: implement <code>backward()</code> for the full computation graph. Test on simple expressions.	Draw the computation graph for a 2-layer net.
W11 Wed	Karpathy: Video 2: “makemore” Part 1 (first 40 min)	Implement a bigram language model from scratch following Karpathy. Type everything yourself.	What is a language model fundamentally computing?

W11 Thu	Karpathy: Video 2 (remaining)	Complete the bigram model. Train it on a names dataset. Generate samples.	What are the limitations of bigrams?
W11 Fri	— Review Day —	FROM SCRATCH: Rebuild micrograd from memory. If you get stuck, note exactly where and revisit.	

Week 12: Consolidation & Phase 2 Capstone

Day	Conceptual	Implementation	Reflection
Mon	Review your study journal from Weeks 7–11. Identify the 3 weakest areas.	Revisit and rebuild the algorithm you struggled with most.	
Tue	Read ISLR Chapter 5 (Resampling Methods) — focus on bootstrap	Implement bootstrap confidence intervals for a regression coefficient.	Why does resampling give us uncertainty estimates?
Wed	Read ISLR Chapter 6 (Regularization overview)	Build a model comparison framework: given a dataset, automatically run linear regression, logistic regression, decision tree, random forest, and your neural network. Output a comparison table.	
Thu	Review: pick 3 StatQuest videos on topics that are still fuzzy	Add early stopping and learning rate scheduling to your neural network.	
Fri	— PHASE 2 CAPSTONE —	End-to-end ML pipeline on a real dataset of your choice. Full EDA, feature engineering, model selection, evaluation. Write a 1-page report on your findings.	Write a reflection: what's your strongest area? Weakest?

Phase 3: Deep Learning, Tokenization, Transformers & Systems

Weeks 13–19 | Thinking in Tensors

Goal: Transition from NumPy to PyTorch. Understand backpropagation at the framework level, build real neural network architectures, and develop fluency with tensors. By end of Phase 3, you should be able to implement a transformer from scratch, understand the tokenization pipeline that feeds it, and have working knowledge of the systems infrastructure that makes training possible.

Resources

- **Karpathy — Neural Networks: Zero to Hero** (Videos 3–7, plus the dedicated tokenization video). This is your primary resource. Karpathy’s pedagogy matches your learning style perfectly—he builds everything from scratch.
- **PyTorch official tutorials** (60-minute blitz) — for syntax reference only, not as a primary learning resource.
- **Deep Learning (Goodfellow et al.)** Chapters 6, 8, 10 — as reference for formal notation when you need it.
- **Karpathy — “Let’s build the GPT Tokenizer”** (YouTube) — the dedicated tokenization video for Week 16.

Discipline Note: 1.5-Hour Sessions

By Week 13, your study habit should be solid. If you haven’t already, **expand to 1.5-hour sessions**: 30 min conceptual, 50 min implementation, 10 min reflection. If 1 hour still feels right, stay there—the habit is more valuable than the extra time.

Week 13: PyTorch Fundamentals & Makemore Part 2

Day	Conceptual	Implementation	Reflection
Mon	PyTorch 60-minute blitz: Tensors section only	Reimplement your linear regression, logistic regression, and 2-layer network using PyTorch tensors (not nn.Module yet). Use torch.autograd for gradients.	How does torch.autograd compare to your micrograd?
Tue	Karpathy: Video 3 “makemore Part 2” (MLP) — first 40 min	Begin implementing the MLP language model following Karpathy. Build the embedding table and hidden layer.	What is an embedding and why is it better than one-hot?
Wed	Karpathy: Video 3 (remaining)	Complete the MLP. Train it. Experiment with hidden size and embedding dimension.	How does increasing hidden size affect training dynamics?
Thu	Goodfellow Chapter 6, Sections 6.1–6.3 (skip proofs, focus on architecture descriptions)	Implement your MLP using nn.Module for the first time. Compare the code structure with your raw-tensor version.	What does nn.Module give you that raw tensors don’t?

Fri	— Review Day —	FROM SCRATCH: Build an MLP classifier for MNIST using <code>nn.Module</code> . Achieve >97% accuracy. Experiment with depth, width, activation functions.	
-----	----------------	---	--

Week 14: Backprop Internals & Activations

Day	Conceptual	Implementation	Reflection
Mon	Karpathy: Video 4 “becoming a backprop ninja” (first 40 min)	Follow along: manually derive and implement backward passes for each operation in the MLP.	Which backward passes are most confusing?
Tue	Karpathy: Video 4 (remaining)	Complete all manual backward pass exercises. Verify each against PyTorch’s autograd.	Why does Karpathy think this exercise is essential?
Wed	Goodfellow Chapter 6.5–6.6 (activation functions, architecture design)	Experiment: train the same MLP with sigmoid, tanh, ReLU, GELU activations. Plot training curves. Examine gradient magnitudes at each layer.	Which activations suffer from vanishing gradients?
Thu	StatQuest: “Batch Norm” + “Dropout”	Add BatchNorm and Dropout to your MNIST MLP. Measure the effect on training stability and test accuracy.	How does BatchNorm affect gradient flow?
Fri	— Review Day —	FROM SCRATCH: Implement BatchNorm forward and backward pass manually (no <code>nn.BatchNorm1d</code>). Verify gradients numerically.	

Week 15: CNNs (Compressed — Not Your Focus, But Required Knowledge)

Day	Conceptual	Implementation	Reflection
Mon	3B1B: “But what is a convolution?”	Implement 2D convolution from scratch: naive nested loops version. Test on a small image with a Sobel edge detection kernel.	What is a convolution computing at each position?
Tue	Goodfellow Chapter 9, Sections 9.1–9.4 (convolution, pooling, variants)	Build a simple CNN for MNIST using <code>nn.Conv2d</code> , <code>nn.MaxPool2d</code> , and <code>nn.Linear</code> . Get >99% accuracy.	How do convolutional layers reduce parameters vs fully connected?
Wed	Karpathy: Video 5 “makemore Part 3” (WaveNet architecture)	Follow along building the WaveNet-inspired architecture. Focus on dilated causal convolutions.	How does the WaveNet architecture relate to sequence modeling?

Thu	Read about adversarial examples: Goodfellow's FGSM paper (2014) — just the intro and method sections	Implement FGSM attack on your MNIST CNN: generate adversarial examples that fool the model. Visualize the perturbations.	Why are neural networks so vulnerable to adversarial perturbations?
Fri	— Review Day —	FROM SCRATCH: Implement a 3-layer CNN for CIFAR-10. Experiment with data augmentation (random crop, horizontal flip) implemented manually.	

Week 16: Tokenization & Transformer Foundations

Theme: Before you can build a transformer, you need to understand the pipeline that feeds it. Tokenization is not a preprocessing afterthought—it's a design decision that shapes everything downstream, from adversarial attacks (GCG operates in token space) to interpretability artifacts (token-level quirks explain many “weird” model behaviors).

Why this addition matters: Your Phase 4 adversarial work and Phase 8 GCG attack both operate directly in token space. Mechanistic interpretability constantly encounters token-level artifacts—knowing exactly how BPE tokenization works gives you the foundation to distinguish genuine model behaviors from tokenization artifacts. Many interpretability “mysteries” dissolve once you understand what the tokenizer actually did to the input.

Resources (Week 16)

- **Karpathy — “Let’s build the GPT Tokenizer”** (YouTube, ~2 hours). The definitive walkthrough of BPE tokenization from scratch.
- **Sennrich et al. — “Neural Machine Translation of Rare Words with Subword Units”** (2016). The original BPE for NLP paper.
- **SentencePiece documentation** — for understanding the alternative to BPE used by many models.

Day	Conceptual	Implementation	Reflection
Mon	Karpathy: “Let’s build the GPT Tokenizer” (first 45 min). Focus on: why character-level is insufficient, the BPE merge algorithm, and how the vocabulary is built from a training corpus.	Implement a basic BPE tokenizer from scratch: (1) start with character-level tokens, (2) count bigram frequencies, (3) merge the most frequent pair, (4) repeat for N merges. Test on a small text corpus. Print the merge table.	Why does subword tokenization outperform both character-level and word-level? What’s the core trade-off?
Tue	Karpathy: “GPT Tokenizer” (remaining ~75 min). Focus on: encoding and decoding, special tokens, the regex-based pre-tokenization used by GPT-2/GPT-4, and the	Complete your BPE tokenizer: add <code>encode(text) → token_ids</code> and <code>decode(token_ids) → text</code> . Implement the regex-based pre-tokenization that splits on whitespace/punctuation boundaries before BPE. Test	What happens to rare words, code, and multilingual text under BPE? How does pre-tokenization affect the merge statistics?

	handling of unknown characters.	roundtrip: <code>assert decode(encode(text)) == text.</code>	
Wed	Read Sennrich et al. (2016) — Sections 1–3. Then explore the SentencePiece library: read the README and understand how unigram language model tokenization differs from BPE.	Implement tokenization analysis tools: (1) given a tokenizer, visualize how a sentence is split into tokens (color-code each token), (2) compute tokens-per-word ratio for different text types (English prose, Python code, URLs, non-Latin scripts), (3) find “tokenization artifacts”—words that split unexpectedly.	How would tokenization artifacts affect a model’s ability to do arithmetic? Spell words? Why does “SolidGoldMagikarp” break models?
Thu	Read about tokenization’s impact on adversarial attacks and interpretability. Skim the GCG paper’s (Zou et al., 2023) discussion of token-space optimization. Read any Neel Nanda blog posts on tokenization artifacts in interpretability.	Adversarial tokenization experiments: (1) find inputs where adding/removing a single character changes the tokenization dramatically, (2) show how these tokenization boundary shifts affect model outputs, (3) implement a simple token-level perturbation attack that exploits tokenization ambiguity.	How does operating in discrete token space differ from operating in continuous embedding space (FGSM/PGD)? What are the implications for adversarial robustness?
Fri	— Review Day —	FROM SCRATCH: Build a complete tokenization pipeline from a blank file: BPE training on a corpus, encoding, decoding, and a visualization tool that shows token boundaries on any input text. Then begin Karpathy’s Video 7 “GPT from scratch” (0:00–30:00) to set up the bigram baseline for next week.	

Weeks 17–18: The Transformer (Two Full Weeks)

This is the most important section of the entire plan. You are building toward mechanistic interpretability research. You must be able to write a transformer from a blank file.

Day	Conceptual	Implementation	Reflection
W17 Mon	Karpathy: Video 7 “GPT from scratch” (0:00–45:00 or continue from Friday’s start)	Implement the Bigram model in PyTorch as Karpathy’s starting point. Integrate your tokenizer from Week 16 as the input pipeline.	What’s the baseline that a transformer needs to beat?
W17 Tue	Karpathy: Video 7 (45:00–1:30:00) — self-attention	Implement single-head self-attention from scratch: Q, K, V projections, scaled dot-product, masking.	What does the attention matrix represent?

W17 Wed	Karpathy: Video 7 (1:30:00–2:00:00) — multi-head attention	Implement multi-head attention: split heads, parallel attention, concatenate, project.	Why multiple heads instead of one big attention?
W17 Thu	Karpathy: Video 7 (remaining) — full transformer block	Implement the full transformer block: multi-head attention + feedforward + layer norm + residual connections.	What does each component contribute?
W17 Fri	— Review Day —	FROM SCRATCH: Write the attention mechanism from a blank file. No references.	
W18 Mon	Read “Attention Is All You Need” (Vaswani et al., 2017) — focus on Section 3	Assemble the full GPT model: token embedding + positional embedding + N transformer blocks + final linear head.	Map Karpathy’s code to the original paper’s notation.
W18 Tue	Goodfellow Chapter 10 (Sequence Modeling) — skim for context on RNNs, then focus on attention	Train your GPT on the tiny Shakespeare dataset. Generate text samples at different training stages.	How does the quality of generation change as training progresses?
W18 Wed	Read the Illustrated Transformer (Jay Alammar’s blog). Read about positional encoding: sinusoidal vs learned.	Add learning rate warmup and cosine decay to your training loop. Compare training curves. Implement both sinusoidal and learned positional encodings. Compare training dynamics.	Why does warmup help transformer training? Why does position information need to be injected?
W18 Thu	Read the GPT-2 paper (Radford et al., 2019) — focus on architecture choices. Read about layer normalization: Pre-LN vs Post-LN.	Implement LayerNorm from scratch (no nn.LayerNorm). Experiment with pre-norm vs post-norm placement. Scale up your model: experiment with depth, width, and more heads. Profile memory usage.	How does LN placement affect training stability? How does scaling each dimension affect performance?
W18 Fri	— Review Day —	FROM SCRATCH: Write the entire GPT model from a blank file. This is the most important exercise in the plan. Then extract attention patterns and write a brief analysis of what you observe—your first mechanistic interpretability project.	Write a 1-page reflection on your deep learning journey so far.

Week 19: Systems & Infrastructure for Deep Learning

Theme: Understanding the engineering that makes training possible. You don’t need to become a systems expert, but knowing how to train efficiently on real hardware makes the difference between running one experiment a day and running twenty.

Why this addition matters: For the “solid AI research engineer” part of your goal, this is a critical practical gap. Every experiment in Phases 5–8 will be bottlenecked by your ability to efficiently use GPU resources. Understanding mixed-precision training, gradient accumulation,

and memory profiling transforms you from someone who trains small toys to someone who can run meaningful experiments on real models. Your hardware background from JHU/APL and CUDA binary analysis experience gives you a head start here.

Resources (Week 19)

- **PyTorch documentation** — `torch.cuda.amp` (automatic mixed precision), gradient checkpointing, memory profiling
- **Hugging Face** — “Efficient Training on a Single GPU” documentation page
- **NVIDIA** — **Mixed Precision Training** whitepaper (Micikevicius et al., 2018) — Sections 1–3
- **Karpathy’s nanoGPT repository** — study the training optimizations in `train.py`

Day	Conceptual	Implementation	Reflection
Mon	Read the NVIDIA Mixed Precision Training paper (Sections 1–3). Focus on: why FP16 is faster, when FP16 breaks (loss scaling), and the master weights pattern. Then read PyTorch’s <code>torch.cuda.amp</code> documentation.	Add mixed-precision training to your Phase 3 transformer: (1) wrap the forward pass in <code>torch.cuda.amp.autocast()</code> , (2) use <code>GradScaler</code> for loss scaling, (3) measure training speed improvement and memory reduction vs FP32. Compare final model quality.	Why can’t we just use FP16 everywhere? What operations are numerically sensitive?
Tue	Read about gradient accumulation and gradient checkpointing. Study the relevant sections in Karpathy’s nanoGPT <code>train.py</code> .	Implement gradient accumulation from scratch: instead of updating weights every batch, accumulate gradients over N mini-batches before stepping. Show that training with <code>batch_size=8</code> accumulated over 4 steps gives identical results to <code>batch_size=32</code> . Then implement gradient checkpointing: use <code>torch.utils.checkpoint</code> to trade compute for memory.	When would you use accumulation vs checkpointing vs both? How does effective batch size affect training dynamics?
Wed	Read Hugging Face’s “Efficient Training on a Single GPU” guide. Focus on: memory profiling, identifying bottlenecks, and the memory breakdown (parameters, gradients, optimizer states, activations).	Build a GPU memory profiler for your transformer: (1) use <code>torch.cuda.memory_allocated()</code> and <code>torch.cuda.max_memory_allocated()</code> to track memory at each training step, (2) break down memory usage by category (model parameters, gradients, optimizer states, activations), (3) visualize memory usage over a training step. Identify the bottleneck.	Where does most GPU memory go during training? How does model size affect each category differently?
Thu	Review Karpathy’s nanoGPT repository: study the training loop	Combine all optimizations into your transformer training pipeline: mixed precision + gradient	How much faster is the fully optimized pipeline? Which

	optimizations (fused AdamW, torch.compile, flash attention). Read about data loading optimization: num_workers, pin_memory, prefetching.	accumulation + optimized data loading + torch.compile (if available). Benchmark the full pipeline: measure throughput (tokens/second) at each optimization level. Create a table showing the cumulative speedup.	optimization gave the biggest single improvement?
Fri	— PHASE 3 CAPSTONE —	Clean up your transformer implementation into a well-documented, modular codebase with all systems optimizations. This becomes your reference implementation. Train a final model with optimal settings and generate samples. Write a brief technical note documenting your optimization results.	This codebase is your foundation for Phases 5–8. Treat it accordingly.

Phase 4: AI Security & Adversarial ML

Weeks 20–23 | Breaking What You Built

Goal: Apply your deep learning knowledge to security-relevant problems. This phase bridges your cybersecurity background with your AI foundations. You'll implement attacks, defenses, and begin developing the adversarial intuition essential for AI security research.

Resources

- **Goodfellow et al. — “Explaining and Harnessing Adversarial Examples”** (FGSM paper)
- **Carlini & Wagner — “Towards Evaluating the Robustness of Neural Networks”** (C&W attack)
- **Madry et al. — “Towards Deep Learning Models Resistant to Adversarial Attacks”** (PGD)
- **PRML Chapters 1–2** — Now you're ready. Use these chapters to formalize your probability and distribution knowledge with Bayesian framing.

Week 20: Adversarial Attacks on Neural Networks

Day	Conceptual	Implementation	Reflection
Mon	Read the FGSM paper (Goodfellow et al., 2014) — full paper, take notes	Implement FGSM attack on your MNIST and CIFAR-10 models. Sweep epsilon values. Plot accuracy vs perturbation magnitude.	Why does the sign of the gradient matter more than the magnitude?
Tue	Read the PGD paper (Madry et al., 2018) — Sections 1–3	Implement PGD (iterative FGSM with projection back to epsilon-ball). Compare attack success rate: FGSM vs PGD at same epsilon.	Why is PGD stronger than FGSM?
Wed	Read about targeted vs untargeted attacks. Skim the C&W paper introduction.	Implement targeted FGSM and PGD: force the model to classify an image as a specific wrong class. Visualize.	What makes targeted attacks harder than untargeted?
Thu	Read about transferability of adversarial examples (Papernot et al., 2016 — intro only)	Train 3 different architectures on MNIST. Generate adversarial examples on Model A. Test if they fool Models B and C.	What does transferability imply about learned representations?
Fri	— Review Day —	FROM SCRATCH: Implement a complete adversarial robustness evaluation: train model, attack with FGSM/PGD at multiple epsilons, generate a robustness curve report.	

Week 21: Adversarial Training & Defenses

Day	Conceptual	Implementation	Reflection
Mon	Read Madry et al. Sections 4–5 (adversarial training)	Implement adversarial training: during each batch, generate PGD adversarial examples and train on them. Compare clean accuracy vs robust accuracy.	What's the accuracy trade-off of adversarial training?
Tue	Read about gradient masking and obfuscated gradients (Athalye et al., 2018 — Sections 1–3)	Implement a “defense” that uses input transformation (JPEG compression, smoothing). Test if FGSM still works. Then implement an adaptive attack that accounts for the defense.	Why do simple input transformations fail as defenses?
Wed	PRML Chapter 1 (Introduction) — Bayesian perspective on learning	Connect adversarial vulnerability to model confidence: visualize the softmax probability distribution on adversarial vs clean examples. Implement temperature scaling.	How does the Bayesian view explain overconfident predictions?
Thu	Read about certified defenses and randomized smoothing (overview only)	Implement randomized smoothing: add Gaussian noise to inputs at inference time, take majority vote. Measure certified radius.	What's the difference between empirical and certified robustness?
Fri	— Review Day —	FROM SCRATCH: Build a full pipeline: train standard model, adversarially train, compare robustness. Write up results.	

Weeks 22–23: Data Poisoning, Model Security & PRML

Day	Conceptual	Implementation	Reflection
W22 Mon	Read about data poisoning attacks: backdoor attacks (Gu et al., 2017)	Implement a backdoor attack: add a trigger pattern to 5% of training images with a target label. Train and test.	How would you detect a backdoor in a model you didn't train?
W22 Tue	Read about model extraction attacks (Tramer et al., 2016 — Sections 1–3)	Implement a model extraction attack: query a “black-box” model with synthetic inputs, train a surrogate model on the responses.	What does your surrogate capture about the original model?
W22 Wed	PRML Chapter 2 (Probability Distributions) — Sections 2.1–2.3	Implement Bayesian linear regression from scratch: prior, likelihood, posterior. Visualize how the posterior updates with data.	How does the Bayesian approach handle uncertainty differently?
W22 Thu	PRML Chapter 2 — Sections 2.3–2.5 (Gaussian distributions, Bayesian inference)	Implement a Bayesian neural network (simple version): replace point weights with distributions. Use MC dropout as an	How does uncertainty differ between clean and adversarial inputs?

		approximation. Show uncertainty estimates on adversarial inputs.	
W22 Fri	— Review Day —	Build a “model security audit” tool: given a trained model, automatically run FGSM, PGD, check for backdoor triggers, and generate a security report.	
W23 Mon–Thu	Read 1–2 papers per day from your research area (attribution circuits, refusal circuits, etc.)	Implement one small experiment inspired by each paper. These don’t need to be complete reproductions—just the core mechanism.	
W23 Fri	— PHASE 4 CAPSTONE —	Write a comprehensive adversarial ML notebook: attacks, defenses, and Bayesian uncertainty analysis on a single model. This is portfolio-quality work.	

Phase 5: ARENA Curriculum, Experimental Methods & Causal Inference

Weeks 24–29 | The Bridge to Research

Goal: You are now equipped to work through the ARENA curriculum independently. This phase transitions you from guided study to self-directed research, while adding the rigorous experimental methodology and causal reasoning foundations that separate good researchers from exceptional ones.

Session Structure: 2 Hours (if sustainable)

By now, if your discipline has built naturally, you may be ready for 2-hour AI sessions: 40 min conceptual, 70 min implementation, 10 min reflection. If 1.5 hours is your ceiling, that's fine—just pace accordingly.

Weeks 24–25: ARENA Chapter 0 — Prerequisites

Day	Activity
W24 Mon	ARENA 0.0: Ray Tracing — this exercises your tensor manipulation skills. You should breeze through the einops/einsum sections given Phase 1–3.
W24 Tue–Wed	ARENA 0.1: Build Your Own Transformer — you've done this. Use this as a validation checkpoint. If you can complete it quickly without AI, your foundations are solid.
W24 Thu–Fri	ARENA 0.2: Build Your Own Training Loop — implement training with wandb logging. Focus on understanding the PyTorch training idioms.
W25 Mon–Wed	ARENA 0.3: ResNets and Model Assembly — build ResNet from the paper. This tests your CNN and residual connection knowledge from Phase 3.
W25 Thu–Fri	ARENA 0.4: Optimization — implement Adam, learning rate scheduling, weight decay from scratch. You did this in Phase 2—this should be review.

Week 26: Experimental Methodology & Statistical Rigor

Theme: When you claim “this circuit is causally responsible for refusal,” you need to know about proper ablation controls, statistical significance with multiple comparisons, confidence intervals on your patching results, and how to avoid p-hacking yourself.

Why this addition matters: The plan builds strong implementation skills but doesn't explicitly teach you how to design rigorous experiments. This is the skill that separates a convincing mechanistic interpretability paper from an unconvincing one. When reviewers question your ARENA Chapter 1 work or your Phase 6 capstone, this is what they're looking for. Your transition from “student” to “researcher” requires this foundation.

Resources (Week 26)

- **Dodge et al. — “Show Your Work: Improved Reporting of Experimental Results”** (2019). The definitive paper on ML experimental reporting.
- **Bouthillier et al. — “Accounting for Variance in Machine Learning Benchmarks”** (2021). On understanding and reporting variance.

- **Wasserstein & Lazar — “The ASA Statement on p-Values”** (2016). The American Statistical Association’s guidance on what p-values actually mean.
- **Benavoli et al. — “Time for a Change: a Tutorial for Comparing Multiple Classifiers Through Bayesian Analysis”** (2017). Bayesian alternatives to null hypothesis testing.

Day	Conceptual	Implementation	Reflection
Mon	Read Dodge et al. (2019) “Show Your Work” — full paper. Focus on: the impact of random seeds on reported results, the distinction between expected validation performance and max validation performance, and their recommendations for reporting.	Take your Phase 2 capstone ML pipeline and re-run it with 20 different random seeds. For each seed, record: final accuracy, best validation accuracy, and training time. Compute: mean, standard deviation, min, max, and 95% confidence interval. Visualize the distribution. How much do results vary?	How different are your best and worst runs? Would you have drawn different conclusions if you’d only run one seed?
Tue	Read the ASA Statement on p-Values (Wasserstein & Lazar, 2016). Then read about multiple comparisons correction: Bonferroni, Holm-Bonferroni, and false discovery rate (Benjamini-Hochberg). Focus on why testing many hypotheses inflates Type I errors.	Implement a statistical testing toolkit from scratch: (1) paired t-test for comparing two models, (2) Bonferroni correction for multiple comparisons, (3) bootstrap confidence intervals for any metric. Apply to your multi-seed experiment: is the difference between your best and second-best model statistically significant?	If you run 100 activation patching experiments, how many will show “significant” results by chance alone at $p < 0.05$? How does this change your interpretation of patching results?
Wed	Read about proper ablation study design. Study examples from top interpretability papers (Wang et al. 2022, Conmy et al. 2023) — focus on their control experiments and how they establish causal claims vs correlational observations.	Design and implement a rigorous ablation template: given a model and a hypothesized circuit, implement: (1) positive control (ablate the circuit, behavior should change), (2) negative control (ablate random non-circuit components, behavior should not change), (3) specificity test (does ablating the circuit affect only the target behavior, not other behaviors?), (4) report with confidence intervals. Test on a simple example.	What makes an ablation convincing? What are the most common ways ablation studies go wrong in interpretability papers?
Thu	Read Bouthillier et al. (2021) on variance in ML benchmarks. Then study how interpretability papers validate their findings: knockdown experiments, consistency across prompts,	Build an experiment manager class that enforces good practices: (1) automatic multi-seed runs with configurable seeds, (2) automatic computation of mean/std/CI for all metrics, (3) logging of all hyperparameters and environment details, (4) automatic generation of results	What’s the minimum number of seeds you need for reliable results in your typical experiment? How would you decide this in practice?

	convergence of different methods.	tables with uncertainty estimates. This becomes your standard tool for all future experiments.	
Fri	— Review Day —	FROM SCRATCH: Reproduce a simple interpretability result (e.g., indirect object identification in a small model) with full statistical rigor: multiple seeds, proper controls, confidence intervals, and a written report that would satisfy a skeptical reviewer. Use your experiment manager.	

Week 27: Causal Inference Foundations for Interpretability

Theme: Your activation patching work is fundamentally causal reasoning—you’re doing interventions and measuring effects. Understanding the formal language of causality gives you more principled tools for designing and validating interpretability experiments.

Why this addition matters: When you do activation patching, you’re performing a do-calculus intervention: you’re asking “what happens if I set this activation to a specific value?” Pearl’s framework gives you the formal language to distinguish interventions from observations, identify confounders in your experiments, and reason about when patching results actually imply causal structure. Geiger et al.’s causal abstraction work connects this directly to circuit analysis, giving you a principled framework for claiming “this circuit implements this computation.”

Resources (Week 27)

- **Pearl — “Causal Inference in Statistics: A Primer”** (2016). Not the full “Causality” textbook—the shorter primer. Chapters 1–3 cover the essential concepts.
- **Geiger et al. — “Causal Abstractions of Neural Networks”** (2021). The key paper connecting causal reasoning to circuit analysis.
- **Geiger et al. — “Inducing Causal Structure for Interpretable Neural Networks”** (2022). Extension to interchange intervention training.
- **Brady Knoll’s “Causal Inference for the Brave and True”** (online textbook, free). Excellent practical introduction.

Day	Conceptual	Implementation	Reflection
Mon	Read Pearl’s Primer, Chapter 1 (Preliminaries and the basics of causal models). Focus on: directed acyclic graphs (DAGs), the distinction between $P(Y X)$ (observation/conditioning) and $P(Y do(X))$ (intervention), and why this distinction matters.	Implement a simple causal DAG library from scratch: (1) define a DAG with nodes and edges, (2) implement d-separation testing (given the DAG, are X and Y conditionally independent given Z?), (3) generate synthetic data from a causal model and verify that conditioning and intervening give different results.	Why does conditioning on a collider create a spurious association? Draw the DAG for your activation patching experiments.
Tue	Read Pearl’s Primer, Chapter 2 (Interventions	Build a connection between causal inference and activation	When you patch an attention head’s output, what are you

	and the do-calculus). Focus on: the truncated factorization formula, the three rules of do-calculus, and the backdoor/frontdoor criteria. You don't need to memorize the rules—focus on the intuition.	patching: (1) draw the causal DAG for a transformer's forward pass (inputs → attention heads → MLPs → residual stream → output), (2) show that activation patching is equivalent to a do-intervention on a specific node, (3) identify potential confounders in your patching experiments (e.g., residual stream carries information from multiple sources).	actually conditioning on vs intervening on? Are there confounders you're not accounting for?
Wed	Read Geiger et al. (2021) "Causal Abstractions of Neural Networks" — Sections 1–4. Focus on: how they define a "causal abstraction" as a mapping between a high-level causal model and a neural network, and how interchange interventions test this mapping.	Implement a simplified interchange intervention experiment: (1) define a high-level causal model for a simple task (e.g., "the model computes greater-than by comparing the magnitude representations at layer L"), (2) perform interchange interventions—swap activations between two inputs at specific model components, (3) check if the model's output changes in the way predicted by the high-level model.	How does the causal abstraction framework improve over just doing activation patching? What does it buy you in terms of confidence about mechanistic claims?
Thu	Read Geiger et al. (2022) on interchange intervention training. Then re-read the activation patching sections from your Phase 5 ARENA notes and Wang et al. (2022) with fresh eyes—now understanding the causal framework underlying the methodology.	Redesign one of your activation patching experiments from ARENA using the causal inference framework: (1) explicitly write the causal DAG, (2) identify what your interventions test and what they don't, (3) identify potential confounders, (4) design additional experiments to rule out alternative causal explanations. Write up the improved experimental design.	How does thinking in terms of do-calculus change how you design interpretability experiments? What experiments were you previously missing?
Fri	— Review Day —	FROM SCRATCH: Build a "causal interpretability toolkit" that combines your causal DAG library with TransformerLens. Given a hypothesized circuit (DAG) and a model, it should: (1) perform interchange interventions to test the hypothesis, (2) compute the causal effect of each component, (3) test for confounders using d-separation, (4) generate a validation report. Test on the IOI circuit.	

Weeks 28–29: ARENA Chapter 1 — Mechanistic Interpretability

This is what everything has been building toward. You’ve already had exposure to this material, but now you have the mathematical foundations, experimental methodology, and causal reasoning framework to engage with it properly.

Day	Activity
W28 Mon–Tue	ARENA 1.1: Transformer Interpretability — TransformerLens basics, activation patching. This time, when you see a hook that extracts residual stream activations, you’ll know exactly what’s being computed because you built the transformer yourself. Apply your Week 26 experiment manager to every result you produce.
W28 Wed–Thu	ARENA 1.2: Indirect Object Identification — Circuit analysis on GPT-2 small. Attention pattern analysis, activation patching, path patching. Frame every patching experiment using Week 27’s causal inference framework. Write the causal DAG. Identify confounders.
W28 Fri	Review and consolidate. Re-read your Phase 3 capstone (attention pattern analysis) and compare with what you’ve now learned about proper circuit analysis. Note the improvements that experimental methodology and causal reasoning bring.
W29 Mon–Wed	ARENA 1.3: Superposition & SAEs — Toy models of superposition, sparse autoencoders. This connects directly to your SAE and Gemma Scope research. Run all experiments with multiple seeds and proper confidence intervals.
W29 Thu–Fri	Begin your own research direction: refusal circuits, attribution circuits, cross-prompt stability. You now have the foundations—mathematical, implementational, methodological, and causal—to do this independently and rigorously.

Phase 6: Information Theory, Representations & Training Dynamics

Weeks 30–34 | The Formal Toolkit for Model Internals

Goal: Master the information-theoretic and representation analysis tools that underpin modern interpretability and safety research, then develop deep understanding of training dynamics—how and when circuits form, why generalization can improve long after memorization plateaus, and how sudden capability jumps happen. These are the daily working vocabulary of AI safety.

Why this phase matters for safety specifically: Alignment research constantly asks “how different are these two models?” and “what information does this layer encode?” Information theory gives you the formal language for these questions. Representation analysis gives you the practical tools. Training dynamics tells you when and how the behaviors you’re studying actually form during the learning process. Without this phase, you can use TransformerLens but you can’t design your own interpretability experiments or reason about how safety-relevant behaviors emerge.

Resources

- **David MacKay — Information Theory, Inference and Learning Algorithms** (free PDF). Chapters 1–4, 8, 28. The gold standard for intuitive information theory.
- **Chris Olah — Blog posts on representations** (colah.github.io). “Neural Networks, Manifolds, and Topology” and “Representing Data” are essential reading.
- **Alain & Bengio — “Understanding intermediate layers using linear classifier probes”** (2016). The foundational probing paper.
- **Kornblith et al. — “Similarity of Neural Network Representations Revisited”** (CKA paper, 2019).
- **3Blue1Brown — “Wordle and Information Theory”** (excellent entropy intuition builder)
- **Nanda et al. — “Progress Measures for Grokking via Mechanistic Interpretability”** (2023). Essential for training dynamics.
- **Power et al. — “Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets”** (2022). The original grokking paper.
- **Belkin et al. — “Reconciling Modern Machine Learning Practice and the Bias-Variance Trade-off”** (2019). The double descent paper.

Week 30: Entropy, KL Divergence & Cross-Entropy

Theme: Every loss function you’ve used is secretly an information theory concept.

Day	Conceptual	Implementation	Reflection
Mon	MacKay Chapter 1–2 (Probability review, Entropy). Watch 3B1B “Wordle and Information Theory” for intuition.	Implement entropy from scratch: $H(X) = -\sum(p \cdot \log(p))$. Compute entropy of coin flips at different biases. Plot H vs p . Implement joint entropy and conditional entropy for 2D distributions.	Why is entropy maximized for a uniform distribution? What does it mean for a distribution to have low entropy?

Tue	MacKay Chapter 2, Section on KL divergence. Read the Wikipedia article for additional examples and properties.	Implement KL divergence: $D_{KL}(P Q)$. Show that it's asymmetric: compute $D_{KL}(P Q)$ and $D_{KL}(Q P)$ for a Gaussian P and a Laplacian Q . Visualize both directions. Show what happens when Q assigns zero probability where P doesn't.	Why is KL divergence not a true distance metric? When does the asymmetry matter in practice?
Wed	MacKay Chapter 2: Cross-entropy. Re-derive cross-entropy loss from an information-theoretic perspective.	Prove to yourself that minimizing cross-entropy $H(P,Q) = H(P) + D_{KL}(P Q)$. Train your logistic regression from Phase 2 and plot: (1) cross-entropy loss, (2) KL divergence between true labels and predictions, (3) entropy of predictions. Show they're related.	Why does the ML community use cross-entropy loss instead of directly minimizing KL divergence? (Hint: they're the same optimization.)
Thu	MacKay Chapter 4: Mutual Information $I(X;Y) = H(X) - H(X Y)$. Read the connection to channel capacity.	Implement mutual information estimation for discrete variables. Given a dataset of (input features, class labels), compute the MI between each feature and the label. Use this as a feature selection method. Compare with correlation.	How does mutual information capture relationships that correlation misses? Can you construct a dataset where correlation is zero but MI is high?
Fri	— Review Day —	FROM SCRATCH: Implement a complete information-theoretic analysis toolkit: entropy, cross-entropy, KL divergence, mutual information, conditional MI. Apply it to analyze the output distributions of your Phase 3 transformer at different layers.	What does it mean that early layers have high-entropy outputs and later layers have low-entropy?

Week 31: Probing Classifiers & What Representations Encode

Theme: You can ask a neural network what it knows by training a tiny model on its internal representations.

Day	Conceptual	Implementation	Reflection
Mon	Read Alain & Bengio (2016) "Understanding intermediate layers using linear classifier probes." Focus on Sections 1–3 and the experimental methodology.	Train a simple 4-layer MLP on MNIST. Extract activations at each layer for the full test set. Train a linear probe (logistic regression) on each layer's activations to predict the digit class. Plot probe accuracy vs layer depth.	At which layer does linear separability emerge? What does this tell you about what each layer computes?
Tue	Read Chris Olah: "Neural Networks, Manifolds, and Topology" (colah.github.io). Focus	Extend Monday's experiment: visualize the activations at each layer using t-SNE or UMAP. Color by digit class. Create a grid	How does the geometry of the representation change? At which point do the classes become separable?

	on the geometric intuition about how networks transform data.	showing how the representation evolves from input through each layer to output.	
Wed	Read about probing for linguistic structure in transformers: Hewitt & Manning (2019) “A Structural Probe for Finding Syntax in Word Representations” — Sections 1–3.	Load a pre-trained GPT-2 small using TransformerLens. Extract residual stream activations at each layer for a set of sentences. Train linear probes to predict: (1) part of speech of each token, (2) whether a token is inside a quoted string, (3) the token’s position in the sentence.	Which layers encode syntactic information? Is it localized or distributed?
Thu	Read about the “linear representation hypothesis”: Park et al. (2023) “The Linear Representation Hypothesis and the Geometry of Large Language Models.” Skim for core claims.	Implement a concept direction probe: given a set of prompts that vary along a single concept (e.g., sentiment, formality, truthfulness), extract activations and find the linear direction that best separates the concepts. Use your SVD/PCA knowledge to find this direction.	If concepts are linearly represented, what does that imply for steering model behavior? How does this connect to refusal circuit research?
Fri	— Review Day —	FROM SCRATCH: Build a general-purpose probing toolkit. Given a model and a labeled dataset, it should: (1) extract activations at every layer, (2) train linear probes, (3) report accuracy by layer, (4) visualize representations with dimensionality reduction. Test on your transformer from Phase 3.	

Week 32: Representation Similarity & Model Comparison

Theme: How do you rigorously compare what two models (or two layers) have learned?

Day	Conceptual	Implementation	Reflection
Mon	Read Kornblith et al. (2019) “Similarity of Neural Network Representations Revisited” — focus on CKA (Centered Kernel Alignment). Sections 1–4.	Implement CKA from scratch: (1) compute the linear kernel $K = XX^T$ for two sets of representations, (2) center the kernels using the centering matrix H , (3) compute HSIC, (4) normalize. Compare layer representations within a single model.	Why is CKA preferred over simple correlation for comparing high-dimensional representations?
Tue	Read about representational similarity analysis (RSA) from neuroscience: Kriegeskorte et al. (2008) — skim for the core idea	Implement RSA: compute pairwise distance matrices for representations at each layer, then correlate these distance matrices across layers. Compare RSA results with CKA results on	How do CKA and RSA agree or disagree? What does each capture that the other misses?

	of comparing representational dissimilarity matrices (RDMs).	the same model. Also implement a cosine similarity matrix between layers.	
Wed	Read about model stitching: Lenc & Vedaldi (2015) or Bansal et al. (2021) “Revisiting Model Stitching.” Skim for the core methodology.	Implement a simple model stitching experiment: train two different models on the same task. Take layers 1–k from Model A and layers k+1–N from Model B. Insert a learned linear transformation between them. Train only the stitching layer. Measure performance. Sweep k.	If stitching works well at layer k, what does that imply about the representations at that layer?
Thu	Read about the “Logit Lens”: nostalgebraist’s blog post. Then read the “Tuned Lens” paper (Belrose et al., 2023) for the improved version.	Implement the Logit Lens on your Phase 3 transformer or GPT-2 small: at each layer, project the residual stream through the unembedding matrix to get a distribution over tokens. Visualize how predictions evolve through the layers for specific prompts.	At which layer does the model “commit” to its final answer? Does it change its mind at intermediate layers?
Fri	— Review Day —	FROM SCRATCH: Build a model comparison dashboard. Given two models trained on the same task: (1) CKA similarity matrix across all layer pairs, (2) RSA analysis, (3) Logit Lens comparison at matched layers, (4) probe accuracy comparison. Test by comparing a standard model with an adversarially trained version from Phase 4.	How does adversarial training change the internal representations?

Week 33: Information Flow in Transformers

Theme: Putting it all together—tracing how information moves through a transformer.

Day	Conceptual	Implementation	Reflection
Mon	Read Elhage et al. (2021) “A Mathematical Framework for Transformer Circuits” — Sections on the residual stream as a communication channel. Focus on how attention heads read from and write to the residual stream.	Implement residual stream decomposition: for a given prompt, decompose the residual stream at each position into contributions from each attention head and MLP layer. Visualize as a stacked bar chart showing which components contribute most to the final prediction.	How does the “residual stream as communication bus” metaphor help you think about circuit analysis?
Tue	Read about activation patching (also called causal tracing): Wang et	Implement activation patching from scratch (no TransformerLens helpers): run	Which components are most causally important? Does the patching result match

	al. (2022) “Interpretability in the Wild” Section 3 on path patching methodology.	the model on a clean prompt and a corrupted prompt. Replace activations at specific (layer, position) pairs from the clean run into the corrupted run. Measure the effect on the output. Sweep all layers and positions to create a patching heatmap.	what the probing results from Week 31 suggested?
Wed	Read Conmy et al. (2023) “Towards Automated Circuit Discovery for Mechanistic Interpretability” — Sections 1–3 on ACDC methodology.	Implement a simplified version of edge-level activation patching: instead of patching entire layers, patch the output of specific attention heads and MLP layers. Build a directed graph of causal importance between components.	How would you validate that a discovered “circuit” is actually the mechanism the model uses, rather than a correlation?
Thu	Review: re-read your notes from Phases 3–6. Write down the top 5 open questions you have about how transformers work internally.	Design and run one small experiment to address your most interesting open question. Use any tools from Phases 3–6.	
Fri	— Review Day —	Comprehensive interpretability analysis of a single behavior in GPT-2 small: (1) identify the behavior with probing, (2) localize it with activation patching, (3) analyze the circuit with attention pattern visualization, (4) validate with ablation using your Week 26 methodology, (5) measure information-theoretic quantities (entropy, MI) at key layers. Write a 2-page research-style report. Apply your causal inference framework from Week 27.	This is PhD application portfolio material. Treat it accordingly.

Week 34: Training Dynamics — Grokking, Double Descent & Phase Transitions

Theme: When you’re studying how circuits form and change, you need to understand when during training features emerge, why generalization can improve long after memorization plateaus, and how sudden capability jumps happen.

Why this addition matters: This is the single most important gap for mechanistic interpretability research. Circuits don’t appear fully formed—they develop through a training process with its own dynamics, phase transitions, and surprises. Understanding grokking tells you that a model might have learned the “right” algorithm long after it appeared to have just memorized. Double descent challenges your basic intuitions about overfitting. Phase transitions explain why capabilities can emerge suddenly at scale. Without this week, you can analyze what circuits exist but you can’t reason about how or when they formed.

Resources (Week 34)

- **Power et al. — “Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets”** (2022). The original grokking discovery.
- **Nanda et al. — “Progress Measures for Grokking via Mechanistic Interpretability”** (2023). The key paper connecting grokking to circuit formation—directly relevant to your research.
- **Belkin et al. — “Reconciling Modern Machine Learning Practice and the Bias-Variance Trade-off”** (2019). The double descent paper.
- **Olsson et al. — “In-context Learning and Induction Heads”** (2022). Phase transitions in transformer training.
- **Neel Nanda — Blog posts on grokking and training dynamics** (neelnanda.io). Excellent companion to the papers.

Day	Conceptual	Implementation	Reflection
Mon	Read Power et al. (2022) “Grokking” — full paper. Focus on: the experimental setup (algorithmic datasets, modular arithmetic), the key finding (generalization occurs long after memorization), and the role of weight decay in enabling grokking.	Reproduce the grokking phenomenon from scratch: (1) create a modular arithmetic dataset (e.g., addition mod 97), (2) train a small transformer on it, (3) track both training loss and test loss over a very long training run (potentially thousands of epochs). Show the characteristic grokking curve where test accuracy jumps from random to perfect long after training loss hits zero.	At what point during training does the model transition from memorization to generalization? What triggers the transition? How would you have interpreted this model if you’d stopped training at the memorization plateau?
Tue	Read Nanda et al. (2023) “Progress Measures for Grokking” — full paper. Focus on: how they use mechanistic interpretability to track circuit formation during training, the “progress measures” that predict grokking before test accuracy improves, and the Fourier analysis of learned representations.	Extend Monday’s experiment with interpretability tools: (1) save model checkpoints during training (every 100 epochs), (2) apply your probing toolkit from Week 31 to each checkpoint—probe for whether the model has learned modular structure, (3) track the formation of interpretable circuits by measuring attention pattern regularity over training time. Can you detect grokking before it shows up in test accuracy?	How do the circuits change during the memorization phase vs the generalization phase? Is there a discontinuous change or a gradual transition? What does this mean for your interpretability research—are you analyzing circuits that are “finished” or still forming?
Wed	Read Belkin et al. (2019) “Reconciling Modern ML Practice and the Bias-Variance Trade-off” — Sections 1–4. Focus on: the interpolation threshold, the double descent curve, and why larger models can be better even when they	Reproduce double descent: (1) train models of increasing size (vary width or depth) on a fixed dataset, (2) plot test error vs model size, showing the classical U-shaped bias-variance curve followed by the second descent, (3) also plot epoch-wise double descent: for a single large model, track test error over training time	How does double descent challenge the classical view of overfitting? What are the practical implications for how you choose model size and training duration in your experiments?

	perfectly fit the training data.	and show the non-monotonic behavior.	
Thu	Read Olsson et al. (2022) “In-context Learning and Induction Heads” — Sections 1–4. Focus on: how induction heads form during training as a phase transition, and how this correlates with the emergence of in-context learning ability.	Implement a phase transition detector for transformer training: (1) train your Phase 3 transformer on a dataset, saving checkpoints frequently, (2) at each checkpoint, measure: in-context learning score (loss on later tokens vs earlier tokens), attention pattern entropy, and specific circuit metrics, (3) look for discontinuous changes in these metrics during training. Do you observe sharp transitions?	How does the concept of phase transitions in training relate to emergent capabilities at scale (Week 36)? If safety-relevant behaviors can emerge through phase transitions, what are the implications for monitoring and evaluation?
Fri	— PHASE 6 CAPSTONE —	Comprehensive training dynamics analysis: (1) reproduce grokking on modular arithmetic, (2) use your full interpretability toolkit to track circuit formation across training, (3) identify the phase transition where the model switches from memorization to generalization, (4) measure information-theoretic quantities (entropy, MI) at key training stages, (5) connect your findings to double descent and scaling phenomena. Write a 3-page research report combining training dynamics with mechanistic interpretability. This bridges your Phase 6 formal tools with your Phase 5 circuit analysis skills.	This capstone is directly relevant to your research direction. The ability to track how circuits form during training is a rare and valuable skill in the interpretability community.

Phase 7: Reinforcement Learning, RLHF, LoRA & the Alignment Pipeline

Weeks 35–38 | How Modern LLMs Are Actually Built

Goal: Understand reinforcement learning from first principles, then follow the full pipeline from pre-training through RLHF to deployment. Master parameter-efficient fine-tuning (LoRA) as the practical tool that makes all alignment experiments feasible. You cannot do meaningful alignment research without understanding how models are trained to be “helpful, harmless, and honest.” This phase fills that gap.

Why this matters: When you study refusal circuits, you’re studying behavior that was instilled during RLHF. When you study jailbreaks, you’re studying failures of RLHF. When you study sycophancy, you’re studying side effects of RLHF. Understanding the training pipeline is prerequisite to understanding the safety-relevant behaviors that emerge from it.

Resources

- **Sutton & Barto — Reinforcement Learning: An Introduction** (free PDF, 2nd ed.). Chapters 1–6, 13.
- **Spinning Up in Deep RL** (OpenAI, spin.openai.com). Excellent practical companion to Sutton & Barto.
- **Ouyang et al. — “Training language models to follow instructions with human feedback”** (InstructGPT paper, 2022).
- **Ziegler et al. — “Fine-Tuning Language Models from Human Preferences”** (2019).
- **Rafailov et al. — “Direct Preference Optimization”** (DPO paper, 2023).
- **Christiano et al. — “Deep reinforcement learning from human preferences”** (2017).
- **Hu et al. — “LoRA: Low-Rank Adaptation of Large Language Models”** (2021). The foundational LoRA paper.

Week 35: RL Foundations — MDPs, Value Functions & Policy Gradient

Theme: RL is decision-making under uncertainty. It’s the framework that turns human preferences into model behavior.

Day	Conceptual	Implementation	Reflection
Mon	Sutton & Barto Chapter 1–2 (The RL Problem, Multi-Armed Bandits). Focus on the agent-environment interface and the exploration-exploitation tradeoff.	Implement the multi-armed bandit problem from scratch: N slot machines with unknown reward distributions. Implement epsilon-greedy, UCB (upper confidence bound), and Thompson sampling strategies. Plot cumulative regret for each.	How does each strategy balance exploration vs exploitation? Which converges fastest?
Tue	Sutton & Barto Chapter 3 (Finite MDPs): states, actions, rewards, transitions, discount factor. Read slowly—this	Implement a simple grid world MDP: define states, actions, transition probabilities, and rewards. Implement value iteration from scratch to find the	What does the discount factor γ do? What happens as $\gamma \rightarrow 0$ vs $\gamma \rightarrow 1$?

	notation is used everywhere in RLHF.	optimal value function $V^*(s)$ and the optimal policy $\pi^*(s)$.	
Wed	Sutton & Barto Chapter 4–5 (Dynamic Programming, Monte Carlo Methods). Focus on the idea of estimating value functions from experience.	Implement Monte Carlo policy evaluation: run your grid world with a fixed policy for 10,000 episodes. Estimate $V(s)$ from the average returns. Compare with the exact solution from value iteration.	Why is Monte Carlo unbiased but high-variance? When is this preferable to dynamic programming?
Thu	Sutton & Barto Chapter 6 (Temporal-Difference Learning): TD(0), SARSA, Q-learning. Focus on the bootstrapping idea.	Implement Q-learning from scratch on your grid world: maintain a Q-table, update with the TD error, use epsilon-greedy exploration. Implement SARSA alongside and compare convergence.	What is the key difference between Q-learning (off-policy) and SARSA (on-policy)? When does it matter?
Fri	— Review Day —	FROM SCRATCH: Implement Q-learning on the OpenAI Gym CartPole or FrozenLake environment. No deep learning yet—just tabular Q-learning. Achieve a reasonable policy.	

Week 36: Deep RL & Policy Gradient Methods

Theme: Policy gradients are the foundation of RLHF. PPO is the specific algorithm used to train ChatGPT.

Day	Conceptual	Implementation	Reflection
Mon	Sutton & Barto Chapter 13 (Policy Gradient Methods): REINFORCE algorithm. Read Spinning Up's "Part 1: Key Concepts in RL" for a parallel explanation.	Implement REINFORCE from scratch in PyTorch: a simple policy network (2-layer MLP) that outputs action probabilities. Train on CartPole. Compute the policy gradient loss: $-\log(\pi(a s)) * G_t$.	Why do we multiply the log-probability by the return? What does this gradient encourage the policy to do?
Tue	Read Spinning Up: "Vanilla Policy Gradient" and "PPO" pages. Focus on the problem of high variance in REINFORCE and how baselines help.	Add a value function baseline to REINFORCE: train a separate network to estimate $V(s)$, use advantage $A(s,a) = G_t - V(s)$ instead of raw return G_t . Compare training curves with and without the baseline.	Why does subtracting the baseline reduce variance without introducing bias?
Wed	Read the PPO paper: Schulman et al. (2017) "Proximal Policy Optimization Algorithms" — Sections 1–3. Focus on the clipped surrogate objective.	Implement PPO from scratch: (1) collect trajectories, (2) compute advantages using GAE, (3) compute the clipped surrogate loss, (4) update policy with multiple epochs of minibatch SGD. Train on CartPole.	What problem does the clipping solve? Why can't we just take large policy gradient steps?

Thu	Read about reward hacking in RL: Amodei et al. (2016) “Concrete Problems in AI Safety” — Section on reward hacking. Then read about specification gaming examples from DeepMind’s list.	Take your PPO agent and deliberately design a flawed reward function that leads to reward hacking. For example, in a simple environment, give reward for “moving” regardless of direction. Document the unintended behavior.	How does reward hacking in RL connect to alignment problems in LLMs? Is sycophancy a form of reward hacking?
Fri	— Review Day —	FROM SCRATCH: Implement PPO end-to-end on a continuous control task (e.g., LunarLander). Compare learning curves: REINFORCE vs REINFORCE+baseline vs PPO.	

Week 37: RLHF, LoRA & Parameter-Efficient Fine-Tuning

Theme: The three-stage pipeline that turns a base LLM into an assistant, plus the practical tool that makes it all feasible on real hardware.

Why LoRA is integrated here: Every alignment experiment you run requires fine-tuning. You’re not going to full-fine-tune anything meaningful on one GPU. LoRA is the tool that makes RLHF, DPO, and all Phase 8 experiments practical. Real RLHF pipelines use LoRA constantly—understanding it is not optional for a research engineer.

Resources (Week 37 Addition)

- **Hu et al. — “LoRA: Low-Rank Adaptation of Large Language Models”** (2021). Short, elegant paper—the key insight is that weight updates during fine-tuning are low-rank.
- **Dettmers et al. — “QLoRA: Efficient Finetuning of Quantized LLMs”** (2023). The memory-efficient extension.

Day	Conceptual	Implementation	Reflection
Mon	Read Hu et al. (2021) “LoRA” — full paper. Focus on: the key insight (weight updates during fine-tuning have low intrinsic rank), the reparameterization $W = W_0 + BA$ where B and A are low-rank matrices, and the efficiency gains. Then read Dettmers et al. (2023) “QLoRA” Sections 1–3.	Implement LoRA from scratch: (1) create a LoRALinear layer that wraps nn.Linear with a low-rank adapter (freeze the original weights, add trainable B and A matrices), (2) implement the forward pass: $x @ (W_0 + B @ A)$, (3) apply LoRA to your Phase 3 transformer’s attention layers only, (4) count parameters: how many trainable vs frozen? Fine-tune on a small dataset and compare with full fine-tuning.	Why does LoRA work? What does the low-rank assumption tell you about what changes during fine-tuning? How does this connect to your representation analysis from Phase 6?
Tue	Read Christiano et al. (2017) “Deep RL from Human Preferences” — the original RLHF formulation. Focus on the	Implement a reward model from scratch using LoRA: (1) take pairs of text completions, (2) encode each with a small transformer, (3) add a	What assumptions does the Bradley-Terry model make about human preferences? When might these fail?

	reward model training from pairwise comparisons.	LoRA-adapted reward head, (4) train with the Bradley-Terry preference loss: $L = -\log(\sigma(r(\text{preferred}) - r(\text{dispreferred})))$. Use synthetic preference data.	
Wed	Read the InstructGPT paper (Ouyang et al., 2022): the three stages of RLHF. Focus on Sections 4–5 on the PPO training stage and the KL penalty.	Implement the full RLHF pipeline with LoRA: (1) SFT stage: fine-tune your transformer on instructions using LoRA, (2) generate completions, score with reward model, (3) compute the KL-penalized reward $r(x,y) - \beta * KL(\pi_{RL} \pi_{SFT})$, (4) update with PPO. Compare LoRA-based RLHF with full fine-tuning in terms of speed and memory.	Why is the KL penalty essential? What goes wrong without it?
Thu	Read about failure modes of RLHF: sycophancy (Perez et al., 2022), reward model overoptimization (Gao et al., 2022 — skim), mode collapse.	Experiment: train your RLHF model with different values of β (the KL coefficient). At low β , observe reward hacking / mode collapse. At high β , observe the model barely changing from SFT. Plot the Pareto frontier of reward vs KL divergence. Use your interpretability tools to compare the representations of the SFT vs RLHF models.	What's the fundamental tension in RLHF? How does β mediate between “do what the reward model says” and “don't stray too far from the base model”?
Fri	— Review Day —	FROM SCRATCH: Diagram the complete RLHF pipeline from pre-training to deployment. For each stage, write down: (1) the training objective, (2) the data required, (3) what behaviors it instills, (4) what can go wrong, (5) how LoRA is used in practice. This should be a comprehensive reference document.	

Week 38: DPO, Constitutional AI & Phase 7 Capstone

Theme: The alternatives to RLHF and the frontier of alignment techniques.

Day	Conceptual	Implementation	Reflection
Mon	Read the DPO paper (Rafailov et al., 2023): the key insight is that the optimal policy under RLHF has a closed-form solution, so you can skip the reward model and PPO entirely. Focus on	Implement DPO training from scratch using LoRA: the loss is $L = -\log(\sigma(\beta * (\log \frac{\pi(y_w x)}{\pi_{ref}(y_w x)} - \log \frac{\pi(y_l x)}{\pi_{ref}(y_l x)})))$. Train on the same preference data as your reward model. Compare the	What are the practical advantages of DPO over PPO-based RLHF? What are the theoretical trade-offs?

	the derivation in Sections 3–4.	resulting model with your RLHF model.	
Tue	Read Bai et al. (2022) “Constitutional AI: Harmlessness from AI Feedback.” Focus on how RLAIIF replaces human feedback with model-generated feedback using a constitution.	Implement a simplified Constitutional AI pipeline: (1) generate responses to harmful prompts, (2) use a separate model (or even your own model with a critique prompt) to evaluate harmlessness, (3) use these AI-generated preferences to train with DPO using LoRA.	What are the risks of using AI feedback instead of human feedback? When might the model learn the wrong thing?
Wed	Read about representation engineering for alignment: Zou et al. (2023) “Representation Engineering: A Top-Down Approach to AI Transparency.” Sections 1–4.	Implement a simple representation engineering intervention: using techniques from Phase 6 (probing, concept directions), find the “honesty” or “refusal” direction in a model’s residual stream. Apply activation addition (steering vectors) to shift behavior at inference time without retraining.	How does representation engineering compare to RLHF as an alignment technique? What are the advantages and risks?
Thu	Read Perez et al. (2022) “Discovering Language Model Behaviors with Model-Written Evaluations.” Focus on using LLMs to generate evaluation datasets for safety properties.	Build a simple automated evaluation pipeline: (1) generate test prompts targeting specific safety properties (refusal, honesty, helpfulness), (2) run your SFT, RLHF, and DPO models on these prompts, (3) score with your reward model, (4) compare in a structured report.	Which alignment technique produces the most robust safety behaviors? Are there systematic differences?
Fri	— PHASE 7 CAPSTONE —	End-to-end alignment experiment using LoRA throughout: start from a small pre-trained language model and apply the full pipeline: SFT → reward model training → RLHF (PPO) vs DPO comparison. Include: (1) training curves, (2) KL divergence tracking, (3) reward model scores, (4) qualitative generation samples, (5) probing analysis comparing representations before and after alignment, (6) LoRA vs full fine-tuning comparison. Write a 3-page research report.	This is serious research experience. You’re now equipped to discuss alignment techniques with real depth.

Phase 8: AI Safety Theory, Red-Teaming & Evaluation

Weeks 39–42 | The Safety Researcher’s Toolkit

Goal: Develop the conceptual frameworks, threat models, and practical evaluation skills that define an AI safety researcher. This phase covers the theoretical foundations of alignment, systematic red-teaming methodology, and the emerging science of AI evaluation. Your cybersecurity background gives you a massive advantage here—red-teaming AI models is structurally similar to penetration testing, and your threat-modeling instincts transfer directly.

Resources

- **Neel Nanda** — “A Comprehensive Mechanistic Interpretability Explainer” (Neel’s blog).
- **Hubinger et al.** — “Risks from Learned Optimization in Advanced Machine Learning Systems” (Mesa-optimization paper, 2019).
- **Anthropic** — “Sleeper Agents: Training Deceptive LLMs That Persist Through Safety Training” (2024).
- **Wei et al.** — “Jailbroken: How Does LLM Safety Training Fail?” (2023).
- **Carlsmith** — “Is Power-Seeking AI an Existential Risk?” (2022).
- **Scaling Laws papers:** Kaplan et al. (2020), Hoffmann et al. (Chinchilla, 2022).

Week 39: Alignment Theory & Threat Models

Theme: Before you can make AI safe, you need a precise understanding of how it could be unsafe.

Day	Conceptual	Implementation	Reflection
Mon	Read Hubinger et al. (2019) “Risks from Learned Optimization” Sections 1–3: the distinction between base optimizers and mesa-optimizers. Understand the taxonomy: aligned, corrigible, deceptive.	Implement a toy mesa-optimization example: train an inner neural network (the mesa-optimizer) that is itself optimizing a proxy objective different from the outer training objective. Train a small RL agent where the reward function changes between training and deployment. Show the agent can learn to optimize for training-distribution reward without generalizing.	What makes mesa-optimization hard to detect? How would you design a test for it?
Tue	Read Carlsmith (2022) “Is Power-Seeking AI an Existential Risk?” — Sections 1–4. Focus on the six premises of the argument and the logical structure.	Build a structured threat model document for AI safety. For each of the following risks, write: (1) concrete scenario, (2) what would need to be true, (3) what evidence would update you toward/away: mesa-optimization, reward hacking, distributional shift, power-seeking, deception.	Which of Carlsmith’s premises do you find most/least plausible? What evidence would change your mind?

Wed	Read Anthropic's "Sleeper Agents" paper (2024): Sections 1–4. Focus on how they train deceptive behavior and how it persists through safety training (SFT, RLHF, adversarial training).	Implement a simplified sleeper agent: train your Phase 3 transformer on a dataset where a rare trigger token causes it to produce different outputs. Then apply SFT on "safe" data and test if the trigger still works.	Does the sleeper behavior persist after fine-tuning? What would it take to remove it? How does this relate to your backdoor attack experience?
Thu	Read about goal misgeneralization: Shah et al. (2022) "Goal Misgeneralization: Why Correct Specifications Aren't Enough for Correct Goals" — Sections 1–3 and the CoinRun example.	Implement the CoinRun-style experiment in a simple grid world: train an agent where the coin is always on the right side of the level. Test in a new level where the coin is on the left. Show that the agent goes right (learned heuristic) rather than toward the coin (intended goal).	How does goal misgeneralization differ from reward hacking? Can you detect it with interpretability tools from Phase 6?
Fri	— Review Day —	Create a comprehensive "AI Safety Threat Model Taxonomy" document that maps: (1) each threat category, (2) the key papers, (3) concrete examples, (4) detection methods, (5) proposed mitigations, (6) connections to your interpretability skills.	

Week 40: Scaling Laws, Emergent Capabilities & Dangerous Capabilities

Theme: Safety research requires understanding how capabilities emerge with scale, because you need to anticipate dangers before they appear.

Day	Conceptual	Implementation	Reflection
Mon	Read Kaplan et al. (2020) "Scaling Laws for Neural Language Models." Focus on the power-law relationships between compute, data, model size, and loss.	Implement scaling law experiments on your Phase 3 transformer: train models at 4–5 different sizes (varying width and depth). Plot validation loss vs parameter count on a log-log scale. Fit the power law $L(N) = aN^b + c$.	Do your results follow the predicted power law? What are the implications for extrapolating to larger models?
Tue	Read Hoffmann et al. (2022, Chinchilla) — Sections 1–3 on compute-optimal training. Then read Wei et al. (2022) "Emergent Abilities of Large Language Models" for the contrasting view.	Compute the Chinchilla-optimal model size and data size for a fixed compute budget. Use your scaling law fits to predict: if you 10x your compute, how should you allocate it? Validate by actually training the predicted configurations.	What does "emergent" actually mean in this context? Is it a real phase transition or a measurement artifact? Read Schaeffer et al. (2023) on "are emergent abilities a mirage" for the counterpoint.

Wed	Read about dangerous capability evaluations: Phuong et al. (2024) “Evaluating Frontier Models for Dangerous Capabilities” or Shevlane et al. (2023) “Model evaluation for extreme risks.” Focus on the evaluation framework.	Design a dangerous capability evaluation suite for a small language model. Include tests for: (1) persuasion/manipulation ability, (2) situational awareness (does the model know it’s being tested?), (3) ability to generate harmful instructions, (4) deception detection. Implement automated scoring for each.	How do you measure dangerous capabilities without giving the model practice at being dangerous? What’s the measurement paradox?
Thu	Read about AI governance and responsible scaling: Anthropic’s Responsible Scaling Policy. Focus on the concept of AI Safety Levels (ASL) and capability thresholds.	Build a “model card” generator: given a trained model, automatically run a suite of evaluations and produce a structured safety report covering: capabilities, limitations, known failure modes, safety evaluations passed/failed, recommended use restrictions.	How would you determine when a model is too dangerous to deploy? What evidence would you need?
Fri	— Review Day —	FROM SCRATCH: Comprehensive scaling analysis. Train 5 model sizes with proper Chinchilla scaling. For each, run your evaluation suite. Plot: capabilities vs scale, safety behaviors vs scale. Identify any “phase transitions” in safety-relevant behaviors. Connect to your Week 34 training dynamics analysis.	

Week 41: Red-Teaming & Jailbreaking LLMs

Theme: Your penetration testing background meets AI. Systematic adversarial evaluation is one of the most practically valuable AI safety skills.

Day	Conceptual	Implementation	Reflection
Mon	Read Wei et al. (2023) “Jailbroken: How Does LLM Safety Training Fail?” Complete paper. Build a taxonomy of jailbreak categories: competing objectives, mismatched generalization.	Implement a systematic jailbreak evaluation framework. Define 10 test cases across categories: (1) direct harmful requests (baseline), (2) role-playing jailbreaks, (3) encoding-based bypasses (Base64, ROT13), (4) context manipulation, (5) prompt injection. Test against an API-accessible model or your own aligned model from Phase 7.	Which categories of jailbreaks are most effective? Is there a pattern to what the model refuses vs. what it complies with?
Tue	Read about automated red-teaming: Perez et al. (2022) “Red Teaming Language Models with	Implement a simplified GCG-style attack: optimize an adversarial suffix by computing gradients of the model’s refusal probability	How does the GCG attack differ from traditional adversarial examples (FGSM/PGD)? Why is the

	Language Models.” Then read about gradient-based jailbreaks: Zou et al. (2023) “Universal and Transferable Adversarial Attacks on Aligned Language Models” (GCG attack).	with respect to the input token embeddings. Use greedy coordinate descent to find tokens that minimize the probability of refusal. Leverage your Week 16 tokenization knowledge here.	discrete token space a unique challenge?
Wed	Read about prompt injection as a security vulnerability: Greshake et al. (2023) “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.”	Implement an indirect prompt injection attack: build a simple “LLM-powered assistant” that reads external documents and answers questions. Hide prompt injection instructions in a document and demonstrate the model following the injected instructions instead of the user’s.	How does prompt injection compare to traditional code injection (SQL injection, XSS) from your cybersecurity background? What are the parallels and differences?
Thu	Read about defense strategies: system prompts, input/output filters, constitutional training, representation engineering for robustness. Compare with traditional security defense-in-depth.	Build a defense-in-depth pipeline for your LLM assistant: (1) input classifier that flags suspicious prompts, (2) output classifier that detects harmful content, (3) representation-level monitoring using probes from Phase 6 to detect when the model’s internal state is “in jailbreak mode.” Test against your jailbreaks from Monday.	Which defense layer catches which attacks? Is there an attack that bypasses all layers?
Fri	— Review Day —	FROM SCRATCH: Write a comprehensive red-team report. Structure it like a penetration test report: (1) executive summary, (2) methodology, (3) findings ranked by severity, (4) reproduction steps, (5) recommended mitigations. Target either your own aligned model or a systematic analysis of a jailbreak category.	

Week 42: Evaluation Science, Benchmarking & Final Capstone

Theme: Measurement is the foundation of science. AI safety needs rigorous evaluation or it’s just vibes.

Day	Conceptual	Implementation	Reflection
Mon	Read about AI evaluation challenges: Liang et al. (2022) “Holistic Evaluation of Language	Implement a multi-dimensional evaluation framework for safety: (1) refusal rate on harmful prompts, (2) false refusal rate on	What’s the right balance between refusal and helpfulness? How do you

	Models” (HELM) — skim the framework section. Then read about Goodhart’s law and how benchmarks get gamed.	benign prompts, (3) helpfulness on standard tasks, (4) consistency (does the model refuse the same prompt rephrased?), (5) calibration (does model confidence match actual safety?). Test on your Phase 7 models.	measure it without the metric being gamed?
Tue	Read about evaluation for specific safety properties: sycophancy detection (Sharma et al., 2023), truthfulness (TruthfulQA), fairness metrics.	Build a sycophancy detector: (1) generate pairs of prompts where the user states an opinion before asking a factual question, (2) compare the model’s answers with and without the stated opinion. Measure the sycophancy rate. Test on your RLHF vs DPO models.	Which alignment technique produces less sycophancy? Does the KL coefficient in RLHF affect sycophancy rates?
Wed	Read about mechanistic analysis of safety behaviors: combine your Phase 6 interpretability tools with your Phase 7–8 safety knowledge. Review your refusal circuit research with Georg.	Comprehensive circuit analysis of a safety behavior: pick one (refusal, truthfulness, or sycophancy). Use probing to localize it, activation patching to validate, attention pattern analysis to understand the mechanism. Apply your full methodological toolkit: causal inference framework, multiple seeds, proper controls.	How does mechanistic understanding of safety behaviors inform better alignment techniques? Can you use circuit-level insights to improve defenses?
Thu	Review: read your study journal from the entire plan. Write a 1-page document: “My Research Agenda” — the 3–5 open questions you most want to investigate.	Design a research proposal for one of your open questions. Include: (1) motivation, (2) hypothesis, (3) methodology with proper experimental design from Week 26, (4) expected results, (5) how you would validate using causal inference from Week 27, (6) potential impact. This is PhD-application-grade writing practice.	
Fri	— PHASE 8 CAPSTONE & PLAN CAPSTONE —	The final capstone combines everything: (1) Take a pre-trained model, (2) align it with RLHF or DPO using LoRA, (3) red-team the aligned model, (4) analyze the attack/defense with interpretability tools, (5) track how safety circuits formed during training (Week 34), (6) propose and implement an interpretability-informed defense. Write a 5-page research report suitable for a workshop	You started 42 weeks ago building vectors from scratch. You now have the skills to do independent, rigorous AI safety research. Write a final reflection on the journey.

		submission. This is your portfolio piece.	
--	--	---	--

Beyond Week 42: The Research Phase

At this point, you are no longer a student following a plan—you are a researcher. Here are the directions that naturally follow from your unique combination of cybersecurity expertise, mechanistic interpretability skills, and alignment knowledge:

Research Directions That Leverage Your Background

- **Interpretability-informed red-teaming.** Use circuit analysis to discover new jailbreak vectors by identifying which model components implement safety behaviors and finding inputs that suppress them. This is the intersection of Phases 6, 7, and 8.
- **Formal verification of safety circuits.** Your VINE project background in verification of neural executables can be applied to verifying that identified safety circuits are robust under distribution shift. Can you prove that a refusal circuit activates for all prompts in a threat class?
- **Reverse engineering RLHF's effect on representations.** Use your Phase 6 representation analysis tools to characterize exactly what RLHF changes in a model's internal representations. Where are safety behaviors stored? Are they distributed or localized? Can they be surgically removed?
- **Scaling laws for safety.** Do safety behaviors emerge, strengthen, or break as models scale? Use your Phase 8 evaluation framework on model families of different sizes to characterize safety scaling laws.
- **Hardware-level safety verification.** Your reverse engineering and embedded systems background (JHU/APL, CUDA binary analysis) positions you uniquely to work on verifying that safety properties are preserved through the compilation and deployment stack—from PyTorch through TensorRT to actual GPU execution.
- **Training dynamics of safety behaviors.** Using your Week 34 training dynamics expertise, study when and how safety-relevant circuits form during RLHF. Do refusal circuits emerge through grokking-like phase transitions? Can you predict when a model will develop deceptive capabilities by monitoring training dynamics?

Ongoing Development

- **Paper reading habit:** Read 2–3 papers per week from arXiv (cs.LG, cs.CL, cs.CR). Focus on papers from Anthropic, DeepMind, OpenAI safety teams, Redwood Research, and MATS scholars.
- **ARENA Chapters 2–3:** Continue with RL (Chapter 2) and language model training (Chapter 3) as needed for your research direction.
- **Conference engagement:** Target NeurIPS, ICML, ICLR safety workshops, and the SaTML (Safety and Trustworthy ML) conference for submissions.
- **Community:** Engage with the Alignment Forum, LessWrong, and the MATS (ML Alignment Theory Scholars) program when applications open.

Appendix A: Resource Quick Reference

Resource	Format	When to Use
3Blue1Brown: Essence of Linear Algebra	YouTube (16 videos)	Phase 1, Weeks 1–3
3Blue1Brown: Essence of Calculus	YouTube (12 videos)	Phase 1, Week 4
Mathematics for Machine Learning (MML)	Free PDF textbook	Phase 1, Chapters 2–7
StatQuest (Josh Starmer)	YouTube channel	Phases 1–2, every algorithm
Andrew Ng: ML Specialization (Course 1)	Coursera	Phase 2 conceptual backbone
ISLR (James, Witten, et al.)	Free PDF textbook	Phase 2, Chapters 2–8
Karpathy: Zero to Hero	YouTube (7 videos)	Phases 2–3, type every line
Karpathy: GPT Tokenizer	YouTube	Phase 3, Week 16 (NEW)
Deep Learning (Goodfellow et al.)	Textbook	Phase 3–4 reference, Ch. 6, 8–10
NVIDIA Mixed Precision Training	Whitepaper	Phase 3, Week 19 (NEW)
PRML (Bishop)	Textbook	Phase 4 Bayesian foundations, Ch. 1–2
ARENA Curriculum	Interactive course	Phase 5
Dodge et al.: Show Your Work	Paper	Phase 5, Week 26 (NEW)
Pearl: Causal Inference Primer	Textbook	Phase 5, Week 27 (NEW)
Geiger et al.: Causal Abstractions	Paper	Phase 5, Week 27 (NEW)
MacKay: Information Theory	Free PDF textbook	Phase 6, Chapters 1–4, 8, 28
Chris Olah's Blog (colah.github.io)	Blog	Phase 6, representation intuition
Nanda et al.: Progress Measures for Grokking	Paper	Phase 6, Week 34 (NEW)
Power et al.: Grokking	Paper	Phase 6, Week 34 (NEW)
Belkin et al.: Double Descent	Paper	Phase 6, Week 34 (NEW)
Sutton & Barto: RL: An Introduction	Free PDF textbook	Phase 7, Chapters 1–6, 13
Spinning Up in Deep RL (OpenAI)	Online tutorial	Phase 7, implementation companion
Hu et al.: LoRA	Paper	Phase 7, Week 37 (NEW)
InstructGPT paper (Ouyang et al., 2022)	Paper	Phase 7, RLHF pipeline
DPO paper (Rafailov et al., 2023)	Paper	Phase 7, DPO implementation
Hubinger et al.: Mesa-Optimization	Paper	Phase 8, deceptive alignment theory
Anthropic: Sleeper Agents	Paper	Phase 8, empirical deception
Carlsmith: Power-Seeking AI	Report	Phase 8, existential risk analysis
Wei et al.: Jailbroken	Paper	Phase 8, jailbreak taxonomy

Appendix B: Weekly Habit Tracker Template

Copy this template each week into your study journal:

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
AI Study (min)							
CPTS Study (min)							
Journal Entry?							
Built Something?							
Energy (1–5)							

Track your energy levels alongside study time. If energy consistently drops below 3 for multiple days, you're overloading. Scale back to the minimum 1-hour session and rebuild. Consistency over intensity, always.

Appendix C: Paper Reading Strategy

Papers appear throughout Phases 4–8. Here is how to read a research paper efficiently in a 30-minute conceptual block:

- **Pass 1 (5 min):** Title, abstract, introduction (first 2 paragraphs), conclusion. You should now know: what they did, why, and what they found.
- **Pass 2 (10 min):** Section headings, all figures and captions, the method section's first paragraph. You should now know: the experimental setup and key results.
- **Pass 3 (15 min):** Read the method section carefully. Skip proofs on first pass. Focus on: what is the input, what is the output, what is the loss function, what are the key hyperparameters.
- **Implementation pass (separate session):** The real understanding comes when you implement the core idea. You don't need to reproduce the full paper—just the central mechanism.

Appendix D: Why Not Kaggle?

Kaggle teaches you to call `sklearn.ensemble.GradientBoostingClassifier()`. Your goal is to be the person who writes `GradientBoostingClassifier`. The Kaggle workflow (load CSV → clean with Pandas → call `model.fit()` → submit predictions) optimizes for leaderboard performance, not for understanding the mathematical mechanics underneath. Every hour spent on Kaggle is an hour not spent implementing backpropagation by hand, which is the skill that actually unlocks ARENA-level interpretability research. The resources in this plan give you the same “accessible warm-up” feeling that Kaggle provides, but they build the right muscle memory: tensors, gradients, and low-level architecture rather than Pandas DataFrames and API calls.

Appendix E: What Changed in the Third Iteration

This iteration adds four new weeks and integrates additional content into two existing weeks, expanding the plan from 38 to 42 weeks. All additions were designed to integrate seamlessly into the existing progression without compressing any original content. Here is a summary of what changed and why:

- **Week 16 — Tokenization (NEW):** Added Karpathy’s dedicated tokenization video and BPE/SentencePiece implementation. Placed at the start of the transformer section because tokenization knowledge is prerequisite to understanding adversarial attacks in token space (Phase 4, Phase 8 GCG attack) and explains many interpretability artifacts encountered in Phases 5–6.
- **Week 19 — Systems & Infrastructure (NEW):** Added mixed-precision training, gradient accumulation, gradient checkpointing, and GPU memory profiling. Placed at the end of Phase 3 because every subsequent phase benefits from efficient training. Leverages the existing hardware/CUDA background.
- **Week 26 — Experimental Methodology & Statistical Rigor (NEW):** Added proper ablation design, multi-seed experiments, statistical testing, and confidence intervals. Placed in Phase 5 between ARENA prerequisites and mech interp content, since this is where the transition from student to researcher happens.
- **Week 27 — Causal Inference Foundations (NEW):** Added Pearl’s do-calculus, d-separation, and Geiger et al.’s causal abstraction framework. Placed immediately before ARENA Chapter 1 because activation patching is fundamentally causal intervention, and this framework improves the rigor of all subsequent circuit analysis.
- **Week 34 — Training Dynamics (NEW):** Added grokking, double descent, and phase transitions in learning. Placed at the end of Phase 6 as a natural extension of information theory and representation analysis. This is the most critical addition for mechanistic interpretability research—understanding when and how circuits form during training.
- **Week 37 — LoRA Integration (EXPANDED):** Added LoRA and QLoRA implementation alongside RLHF. Integrated rather than standalone because LoRA is the practical tool that makes all alignment experiments feasible on realistic hardware. Every RLHF, DPO, and evaluation experiment from Phase 7 onward now uses LoRA.